

DOCUMENTO DESTINADO À BANCA EXAMINADORA

flySOP / SOPADMIN

Guia Técnico da Aplicação

Análise do código-fonte e confronto com o plano *virtual-jingling-cookie.md*.

Versão: 1.0 | **Data:** 10/08/2026

Stack: Laravel 8.83 · PHP 8.1 · PostgreSQL · Sanctum · AdminLTE · Vue 2

Este documento resulta da análise do código em `flysop/` e do plano `virtual-jingling-cookie.md`.

Não inventa funcionalidades. Diferencia claramente: implementado · parcial · planejado.

Sumário

1. Apresentação 2. Visão geral da solução 3. Tecnologias utilizadas 4. Arquitetura da aplicação 5. Estrutura do projeto 6. Funcionalidades implementadas 7. Funcionalidades previstas no `virtual-jingling-cookie.md` 8. Fluxos principais da aplicação 9. Regras de negócio 10. Banco de dados 11. APIs e integrações 12. Segurança 13. Tratamento de erros e validações 14. Interface e experiência do usuário 15. Testes 16. Build, execução e implantação 17. Desempenho e escalabilidade 18. Limitações conhecidas 19. Roadmap 20. Conclusão 21. Glossário 22. Anexos

1. Apresentação

O **flySOP** (SOP / SOPADMIN) é uma aplicação web monolítica para gestão de **ocorrências públicas**, com painel administrativo, área do motorista/agente de campo, site institucional e API para clientes.

Problema que resolve: coordenar abertura, atribuição, deslocamento e acompanhamento de ocorrências georreferenciadas entre múltiplos atores (administração, motorista e cliente), com visão em mapa e atualização de posição.

Objetivo: centralizar o cadastro e a operação de ocorrências, órgãos, tipos/status, motoristas e controle de acesso, com suporte a GPS e canal em tempo real para a rota do motorista vinculada a uma ocorrência.

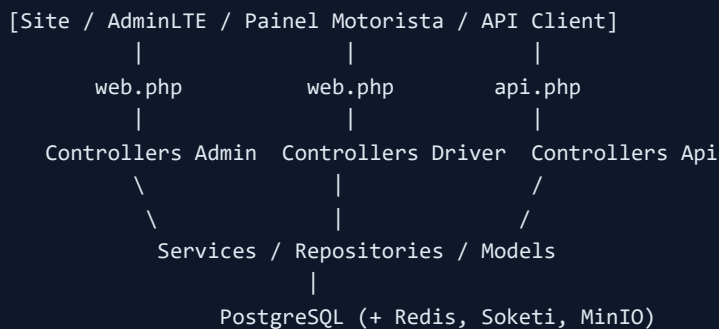
Usuários: administradores/operadores (AdminLTE); motoristas autenticados (`ensure.driver`); clientes via API Sanctum; visitantes do site público.

Contexto: solução multi-tenant (`tenant_id`), evoluída a partir de um núcleo SaaS (planos/categorias/produtos ainda presentes) e do domínio de ocorrências/motoristas. O plano `virtual-jingling-cookie.md` propõe a evolução para "Central Inteligente" (SLA, despacho, dashboard avançado, LGPD etc.), **mantendo o multi-tenant como está, sem expansão**.

Benefícios já observáveis no código: CRUD operacional de ocorrências; painel do motorista com aceite/recusa/status; gravação unificada de posição com broadcast; mapa no admin; ACL por permissões; API autenticada para consultas.

2. Visão geral da solução

A aplicação funciona como monólito Laravel: o navegador consome Blade (admin/driver/site) e JSON (dashboard/API); o backend aplica FormRequests, Services e Repositories nos domínios maduros; o PostgreSQL persiste os dados; Soketi/Pusher transporta eventos de posição.



Principais módulos (código atual)

- Autenticação web (laravel/ui) e API (Sanctum para client)
- Admin: dashboard, ocorrências, imagens, motoristas, tipos/status/órgãos, tenants, users, ACL
- Driver: dashboard, ocorrências atribuídas, aceite/recusa/status, POST de posição
- Site: landing institucional
- Realtime: evento DriverPositionUpdated → canal privado occurrence.{id}
- Legado SaaS: categories, products, tables, profiles/plans (views/migrations)

3. Tecnologias utilizadas

Tecnologia	Finalidade	Onde é utilizada
PHP 8.1 / Laravel 8.83	Backend monolítico	app/, Dockerfile.dev
PostgreSQL 15	Banco principal	serviço pgsql no Compose
MySQL 8	Serviço auxiliar no Compose	serviço mysql
Redis + predis	Fila (e potencial cache)	redis + worker queue
Laravel Sanctum	Auth API por token	model Client, routes/api.php
laravel/ui	Login/registro web	controllers Auth/*
AdminLTE	UI do painel	views admin
Vue 2 + Laravel Mix	Ilhas JS (Echo/Tracker)	resources/js/
Vite 5	Assets site/CSS	vite.config.mjs
Pusher PHP + Echo + pusher-js	Broadcasting	Events, bootstrap.js
Soketi	WebSocket self-hosted	Compose porta 6001
Google Maps JS	Mapas/Places	dashboard e formulário
AWS S3 / MinIO	Arquivos	Flysystem / Compose
Laravel Telescope	Observabilidade	pacote + migrations
Docker Compose	Ambiente local	docker-compose.dev.yml
Fly.io / Vercel / Procfile	Deploy	fly.toml, vercel.json
PHPUnit 9	Testes (mínimos)	tests/

4. Arquitetura da aplicação

4.1 Backend

Duas realidades coexistentes (docs/specs/architecture.md):

1. **Camadas completas:** Controller → FormRequest → Service → Repository → Model (Occurrence, Tenant, TypeOccurrence, Client, ClientConsumer, Dashboard, DriverPosition). 2. **CRUD simples:** Controller injeta Model (Category, Product, Table, User, Role, Profile, Permission, Issuing, StatusOccurrence, Driver).

4.2 Frontend

Blade + AdminLTE no admin; Blade no driver e site; Vue 2 pontual (DriverTracker.vue); JS para Google Maps no dashboard/formulário.

4.3 API

routes/api.php + Resources. Sanctum para leituras sensíveis. **POST /api/occurrences** **permanece público** no código atual.

4.4 Autenticação e autorização

- Web: sessão + auth ; motorista: auth + ensure.driver
- API: auth:sanctum (e residual auth:api)
- ACL: Roles/Permissions + Gates em AuthServiceProvider
- isAdmin() via allowlist de e-mails em config/acl.php (**não** Role "Administrador")
- Policies Laravel: **não implementadas** (\$policies vazio)

4.5 Multi-tenant

ManagerTenant + TenantScope / TenantTrait (Category, Product, Table). Demais domínios frequentemente filtram tenant_id de forma explícita.

4.6 Tempo real

DriverPositionService::record() persiste e, se houver occurrence_id, faz broadcast(new DriverPositionUpdated). Autorização do canal em routes/channels.php.

5. Estrutura do projeto

```
flysop/
├── app/
│   ├── Console/Commands/      # driver-positions:clean
│   ├── Events/                # DriverPositionUpdated
│   ├── Http/Controllers/{Admin,Api,Driver,Site,Auth}/
│   ├── Models/
│   ├── Repositories/ (+ Contracts/)
│   ├── Services/
│   ├── Tenant/
│   └── Providers/
├── routes/{web,api,channels}.php
├── resources/views/{admin,driver,site,auth}/
├── resources/js/              # DriverTracker.vue, occurrenceMap.js
├── database/migrations|seeders
├── docker/ + docker-compose.dev.yml
├── docs/specs/
├── tests/                     # apenas ExampleTest
├── fly.toml | vercel.json | Procfile
└── public/
```

6. Funcionalidades implementadas

6.1 Autenticação web

Login/logout/registo/recuperação via `laravel/ui`. `HomeController` redireciona ao admin. Arquivos: `app/Http/Controllers/Auth/*`.

6.2 Autenticação API (Client)

`POST /api/sanctum/token`, `GET /api/auth/me`, `POST /api/auth/logout`. Model `Client` com `HasApiTokens`.

6.3 Cadastro Client / ClientConsumer

`POST /api/client`, `POST /api/clientConsumer`, `GET /api/client/{id}` com `Services/Repositories`.

6.4 Dashboard administrativo

`DashboardService::getStats()` (contagens básicas); JSON `occurrencesRecent` e `driversLastPositions`; mapa Google com markers e **polling**. **Não inclui** KPIs de SLA, Chart.js, heatmap/cluster do plano.

6.5 Gestão de ocorrências (Admin)

CRUD + search; anexos; atribuição de motorista; `driverRoute`. Camadas: `OccurrencesController`, `OccurrenceService`, `OccurrenceRepository`, `views admin/pages/occurrences`.

6.6 Painel do motorista

Middleware `ensure.driver`; listagem/detalhe; `accept / reject / updateStatus`. Controllers em `app/Http/Controllers/Driver/`.

6.7 GPS e tempo real

POST /driver/position (throttle 20/min) → DriverPositionService → driver_positions → DriverPositionUpdated . Front: DriverTracker.vue . Retenção: php artisan driver-positions:clean .

6.8 Cadastros auxiliares

TypeOccurrence, StatusOccurrence, Issuing, Driver, Tenant, User, Category, Product, Table, Permission, Profile, Role + pivôs ACL.

6.9 Site público

SiteController@index — landing SOP.

6.10 Infraestrutura local

Docker: app, nginx, pgsql, mysql, redis, queue, soketi, minio. Telescope instalado.

7. Funcionalidades previstas no virtual-jingling-cookie.md

Contagem (itens do plano analisados)

Status	Quantidade
✔ Implementado	4
● Parcialmente implementado	11
● Planejado / não implementado	28
Total	43

Tabela de confronto plano × código

Funcionalidade	Prioridade	Status	Evidência
Unificar gravação de posição (DriverPositionService)	P0	✓	app/Services/DriverPositionService.php
Remover/proteger API insegura de localização	P0	✓	DriverLocationController removido
Restringir canal occurrence.{id}	P0	✓	routes/channels.php
Broadcasting real (Pusher/Soketi)	P0	●	Soketi no Docker; Fly ainda documenta sync/log
Fila Redis + worker	P0	●	Compose redis + queue ; Fly pendente
auth:sanctum em endpoints sensíveis	P0	●	GETs protegidos; POST /api/occurrences público
Campo protocol único	P1	●	sem migration
priority_id e due_at (SLA)	P1	●	sem migration
Tabela priorities + CRUD	P1	●	não existe
sla_hours / parent_id em tipos	P1	●	ausente
occurrence_status_history	P1	●	não existe
Endereço estruturado	P2	●	apenas address livre
FormRequests/views protocolo/timeline	P2	●	não implementado
Seeder novos Roles do plano	P1	●	ACL genérico; perfis do plano não seedados
Laravel Policies	P1	●	\$policies vazio
isAdmin por Role Administrador	P1	●	allowlist config/acl.php
departments e teams	P2	●	não existe
Menu filtrado por permissão	P2	●	Gates existem; menus dos novos perfis não
Remover/reavaliar Profile legado	P3	●	Profile ainda no CRUD
Status expandido + is_terminal / order	P1	●	CRUD sem fluxo expandido
Validação de transições	P1	●	ausente no Service
Timeline na tela	P1	●	depende do histórico
Auditoria ampla	P2	●	sem activitylog
Evidências antes/depois	P2	●	imagens existem sem phase /geo
Notificações internas	P2	●	não implementadas
KPIs operacionais (SLA)	P1	●	só contagens básicas
Gráficos Chart.js	P2	●	não instalado
Clustering e heatmap	P2	●	mapa básico
Filtros avançados	P2	●	search admin parcial
Busca global FTS	P2	●	sem tsvector
Push no dashboard	P1	●	Echo no Tracker; dashboard faz polling

Ranking haversine / despacho	P2	●	não existe
Sugestão com confirmação humana	P2	●	atribuição manual CRUD
Limite de rota + retenção	P3	●	driver-positions:clean existe
Sync status Driver↔Occurrence	P2	●	independentes
Fechar duplicação de posição	P1	✓	coberto na Fase 0
cube / earthdistance	P3	●	opcional futuro
Exportação PDF	P2	●	não no composer
Exportação Excel	P2	●	não no composer
Geração via fila (relatórios)	P2	●	jobs inexistentes
Deteção de duplicidade	P3	●	não implementado
LGPD completa	P2	●	retenção de posições; demais itens não
2FA/MFA	P3	●	não implementado

Interpretação

A **base operacional** (ocorrências, motorista, GPS, ACL) está utilizável. A **Fase 0** avançou em segurança de posição/canal e preparou Redis/Soketi no Docker. As **Fases 1–6** do MVP “Central Inteligente” estão, em sua maioria, **planejadas**.

8. Fluxos principais da aplicação

8.1 Acesso admin

Login web → sessão → dashboard → CRUDs conforme menu/permissions.

8.2 Criação e atribuição

Admin cria ocorrência (`OccurrenceService::storeForAdmin`) → anexa imagens → associa `driver_id` → motorista visualiza em `/driver`.

8.3 Aceite e andamento

Motorista autenticado + `ensure.driver` → `accept/reject/updateStatus` → envia posições → broadcast → admin acompanha no `DriverTracker` /mapa.

8.4 API cliente

Token Sanctum → consulta `tenants/ocorrências/tipos` → pode criar via `POST /api/occurrences` (público hoje).

9. Regras de negócio

- Broadcast de GPS somente com `occurrence_id` e ocorrência atribuída ao motorista (`DriverPositionService`).

- Canal `occurrence.{id}`: admin / permissão `occurrences` / motorista dono.
- Painel driver exige vínculo User↔Driver.
- `Gate::before` libera quase tudo para `isAdmin()`, exceto `driver.panel`.
- Criação admin preenche `users_id` e tenta resolver `clients_id` do tenant.
- **Não encontrado:** protocolo, SLA, máquina de estados, haversine, merge de duplicatas.

10. Banco de dados

Tecnologia: PostgreSQL (principal).

Tabela	Papel
<code>tenants</code>	Organização
<code>users</code>	Login web
<code>occurrences</code>	Ocorrência
<code>occurrences_imagens</code>	Anexos
<code>drivers</code>	Agente de campo
<code>driver_positions</code>	Trilha GPS
<code>type_occurrences</code> / <code>status_occurrences</code> / <code>issuings</code>	Domínio auxiliar
<code>roles</code> / <code>permissions</code> (+ pivôs)	ACL
<code>clients</code> / <code>client_consumers</code>	API
<code>categories</code> / <code>products</code> / <code>tables</code> / <code>plans</code>	Legado SaaS

Ausentes (plano): `priorities`, `occurrence_status_history`, `departments`, `teams`.

```
Tenant 1-* User
Tenant 1-* Driver 1-* DriverPosition
User 0..1-1 Driver
Occurrences *-1 Type/Status/Issuing
Occurrences 0..1-* DriverPosition
Occurrences 1-* OccurrencesImagens
Occurrences *-0..1 Driver
```

11. APIs e integrações

Método	Endpoint	Descrição	Auth
POST	/api/sanctum/token	Token client	Público
GET	/api/auth/me	Perfil	sanctum
POST	/api/auth/logout	Logout	sanctum
POST	/api/client	Registra client	Público
POST	/api/clientConsumer	Registra consumer	Público
GET	/api/client/{id}	Exibe client	Público
POST	/api/occurrences	Cria ocorrência	Público
GET	/api/occurrences	Lista	sanctum
GET	/api/occurrences/{uuid}	Detalhe	sanctum
GET	/api/occurrences/getOccurrenceByClientId/{id}	Por client	sanctum
GET	/api/tenants	Lista	sanctum
GET	/api/tenants/{uuid}	Detalhe	sanctum
GET	/api/typeOccurrence	Tipos	sanctum
POST	/driver/position	GPS	auth + ensure.driver

Integrações: Google Maps; Pusher/Soketi; AWS S3/MinIO; Fly.io/Vercel.

12. Segurança

- Hash Laravel; CSRF web; throttle na posição.
- Sanctum em leituras API; criação pública de ocorrência = gap.
- Canal privado restringido (Fase 0).
- API insegura de localização removida.
- Admin por allowlist de e-mail (dívida vs plano).
- Secrets apenas via .env (não reproduzidos neste documento).

13. Tratamento de erros e validações

FormRequests StoreUpdate*; InvalidArgumentExceção no GPS; abort(404) em finds; flash/JSON conforme camada. Sem Problem+JSON uniforme nem erros de SLA.

14. Interface e experiência do usuário

- **Admin:** AdminLTE (cadastros, ocorrências, motoristas, ACL, dashboard/mapa).
- **Driver:** layout reduzido (lista, detalhe, status, GPS).

- **Site:** landing responsiva.
- **Componente Vue:** `DriverTracker.vue`.
- Screenshots não incluídos automaticamente neste documento.

15. Testes

Apenas `tests/Unit/ExampleTest.php` e `tests/Feature/ExampleTest.php`. **Sem** testes de domínio nem cobertura comprovada.

```
php artisan test
# ou
./vendor/bin/phpunit
```

16. Build, execução e implantação

Pré-requisitos

Docker + Compose (WSL recomendado) **ou** PHP 8.1 + Composer + Node 18+ + Postgres.

Execução Docker

```
cp .env.docker.example .env
docker compose -f docker-compose.dev.yml up -d --build
# http://127.0.0.1:8000
docker compose -f docker-compose.dev.yml exec app php artisan db:seed
```

Configuração

Definir `APP_KEY`, `DB_`, `REDIS_`, `BROADCAST_DRIVER / PUSHES_`, `GOOGLE_MAPS_API_KEY`, `AWS_`. **Não incluir secrets neste guia.**

Build front

```
npm install
npm run dev
npm run build
```

Deploy

- Fly.io (`fly.toml` — worker/broadcast em produção ainda a consolidar)
 - Vercel PHP (`vercel.json`)
 - Procfile Apache/Heroku-style
-

17. Desempenho e escalabilidade

Paginação nas listagens; fila Redis no Docker; throttle GPS; limpeza de posições antigas; polling no dashboard. Haversine/PostGIS/FTS do plano **não** implementados.

18. Limitações conhecidas

- Fases 1–6 majoritariamente planejadas
 - `POST /api/occurrences` público
 - Admin por e-mail allowlist; sem Policies
 - Dashboard sem SLA/gráficos/heatmap
 - Dualidade Mix+Vite; stubs (`MailService` , `CategoryProduct`)
 - Testes de domínio ausentes
 - `StoreUpdateOccurrences` duplicado em dois namespaces
 - Produção Fly: fila/broadcast ainda limitados conforme `fly.toml`
-

19. Roadmap

Horizonte	Conteúdo
Já avançado	Base ocorrências+motorista+ACL; <code>DriverPositionService</code> ; canal privado; Redis/Soketi/queue no Docker
Parcial	Broadcast/fila produção; auth API completa; dashboard/mapa avançados; evidências; retenção GPS
Próximos (P1)	Protocolo/SLA/histórico; Policies + roles; KPIs; push no dashboard
Evoluções (P2/P3)	Despacho haversine; Chart.js/cluster/heatmap; FTS; PDF/Excel; duplicidade; LGPD; 2FA
Fora do MVP	App nativo, WhatsApp/SMS, PostGIS avançado, IA

20. Conclusão

O flySOP apresenta **base técnica sólida e utilizável** para gestão de ocorrências com admin, motorista, geolocalização e início de tempo real, em arquitetura Laravel documentada.

O plano *virtual-jingling-cookie.md* descreve a Central Inteligente completa. A análise mostra **Fase 0 parcial com ganhos reais de segurança** e **Fases 1–6 predominantemente planejadas**. Para a banca: há produto operacional + arquitetura clara + roadmap explícito, com transparência sobre o que está no repositório e o que ainda é evolução.

21. Glossário

Termo	Significado
Ocorrência	Registro de evento/atendimento
Tenant	Organização no multi-tenant
Sanctum	Auth por token da API
Broadcasting	Eventos via WebSocket
Soketi	Servidor WS (protocolo Pusher)
ACL	Controle de acesso
SLA	Prazo-alvo (plano)
Haversine	Distância geográfica (plano)
AdminLTE	Template do painel

22. Anexos

A. Métricas do plano

- Itens analisados: **43**
- Implementados: **4**
- Parciais: **11**
- Planejados: **28**

B. Principais módulos documentados

Autenticação web/API · Dashboard · Ocorrências · Motorista · GPS/Realtime · Cadastros/ACL · Site · Docker/Infra

C. Referências

- `virtual-jingling-cookie.md` (plano)
- `docs/specs/architecture.md`, `modules.md`, `security.md`, `backend.md`
- `README.DOCKER.md`, `docker-compose.dev.yml`, `fly.toml`

Documento gerado por análise do código-fonte. Não contém senhas, tokens ou chaves privadas.